

Artificial Intelligence (AI) - Introduction

Dr. K. K. THYAGHARAJAN
Professor & Dean (Research)
R.M.D. Engineering College

Scan the QR code and it will provide the link to all videos. It is a trusted link and safe.



<https://www.youtube.com/c/KKTVirtualLearning>

22EC913 ARTIFICIAL INTELLIGENCE AND MACHINELEARNING

UNIT I INTRODUCTION TO ARTIFICIAL INTELLIGENCE 9

Definitions - Importance of AI, Evolution of AI - Applications of AI, ~~Classification of AI systems with respect to environment.~~ Introduction to Python - Basic Libraries in Python (Pandas, Numpy, Matplotlib) - Conditional- Iterative Statements and Function - Basic Data Exploration - Advanced Functions for Data Manipulation.

Explain about the classification of AI systems with respect to environment. (End Sem)

Describe the Advanced Functions for Data Manipulation in python. (End Sem)

Machine Learning <https://huggingface.co/kkthyagharajan>

UNIT II INTRODUCTION TO MACHINE LEARNING 9

Introduction to Machine Learning - Types of Machine Learning - Supervised and Unsupervised - Data exploration - Target Variables, Independent Numerical Variables, Categorical Variables - Splitting of Data - ~~Feature Scaling of Data~~

UNIT III INTRODUCTION TO DATA SCIENCE

9

Introduction to Data science - Introduction to Statistics Central Tendency - Data Distribution - Probabilities of Discrete and Continuous Variables- Introduction to Inferential Statistics - Hypothesis Testing - T tests - Chi Squared Tests Correlation. Understanding the types of Predictive Models - ~~Univariate Analysis - Bivariate Analysis - Treating Missing Values - How to treat Outliers - Transforming the Variables - Basics of Model Building.~~

UNIT IV LINEAR REGRESSION AND LOGISTIC REGRESSION

9

Linear Regression - Introduction to Linear Regression, Gradient Descent, Feature Engineering - Building First Predictive Model using Regression and Evaluate Performance. Logistic Regression - Basics of Logistic Regression, Evaluation Metrics. ~~Implementing Logistic Regression~~

UNIT V DECISION TREE, ENSEMBLE MODELS AND CLUSTERING

9

Introduction to Decision Tree - Improving Model Performance by Pruning/Hyperparameters Tuning. Basics of Ensemble Techniques - Random Forest - Implementation of Bagging and Random Forest. Clustering - Understanding K-means - Implementation of K-means.

TEXT BOOKS:

1. **Andrew Ng, Machine Learning Yearning, 2018.**
2. Poole, D. and Mackworth, A., Artificial Intelligence: Foundations of Computational Agents, Cambridge University Press, 2010.

REFERENCES:

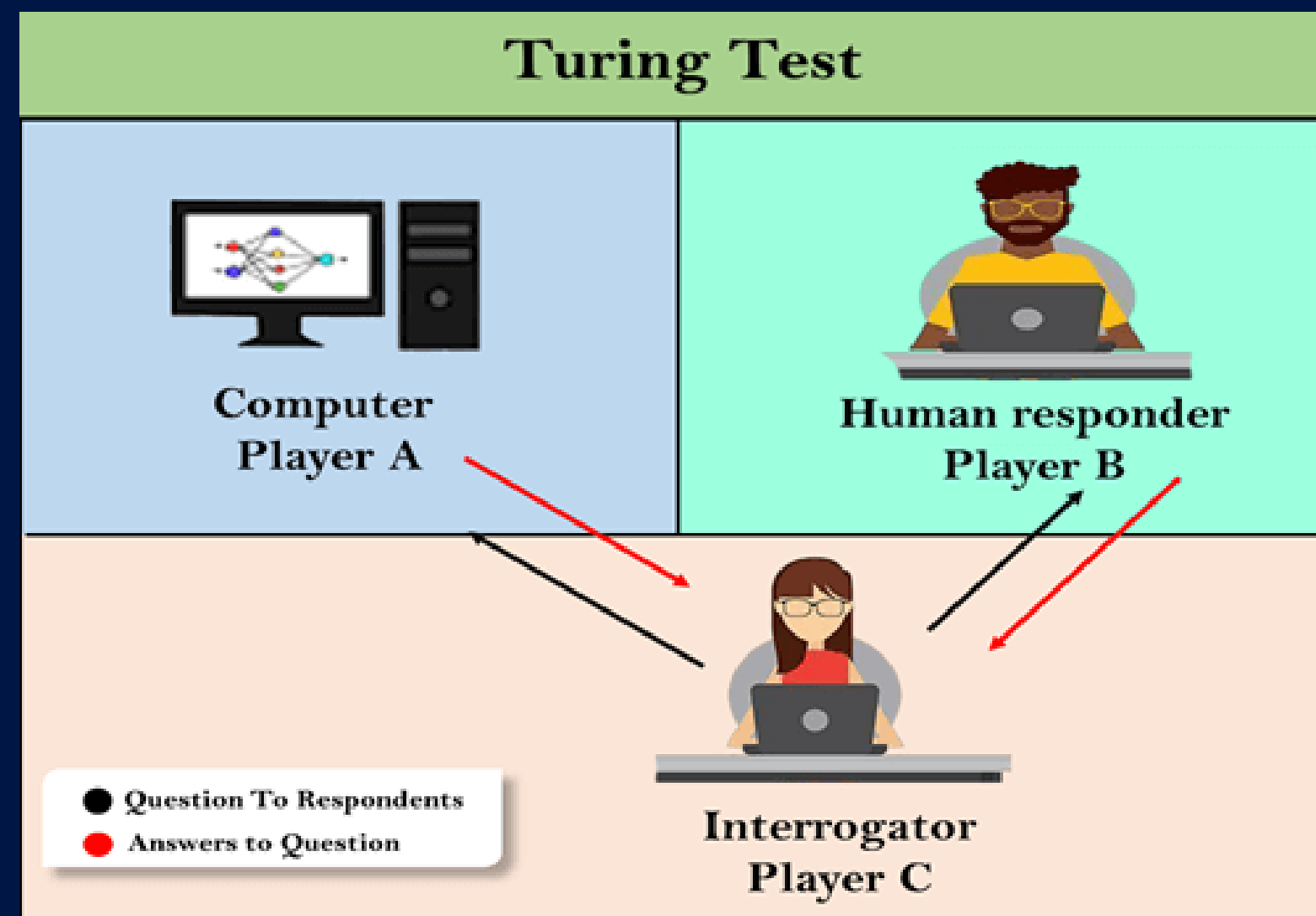
1. Sebastian Raschka and Vahid Mirjalili, Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow, 2nd Edition, Packt Publishing, 2017.
2. Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, MIT Press, 2012.
3. **Alpaydin, E., Introduction to Machine Learning. 3rd edition, The MIT Press, 2020**
4. Trevor Hastie, Robert Tibshirani, and Jerome Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Springer, 2009.
5. Russell, S. and Norvig, P. 2015. Artificial Intelligence - A Modern Approach, 3rd edition, Prentice Hall.

UNIT 1

Artificial Intelligence (AI) - Simulating human intelligence using machines or systems

Machine Learning – Making machines to learn from data and improve their performance over time without being explicitly programming.

Machine Learning is a subset of AI



AI Definitions

AI as a Science: According to John McCarthy, the Father of AI, AI is the Science and Engineering of Making Intelligent machines or Intelligent computer programs.

Artificial Intelligence (AI) - Creating machines or systems that can **simulate human intelligence** often in specialized domains

AI is a system that has **understanding, learning, reasoning, problem-solving and decision making capabilities** which are important in human intelligence. often in **specialized domains**.

Machine Intelligence: AI involves the **development of algorithms, models, and systems** that enable machines to exhibit behaviors that **mimic cognitive functions** associated with human minds.

Problem-Solving Systems: AI involves designing systems that can **interpret data**, learn from it, and use that learning to adaptively solve problems, make decisions, or achieve specific goals.

Automation and Rational Agents: It is about creating **autonomous entities or rational agents** that perceive their environment and take actions to maximize the chances of successfully achieving their goals.

AI can be further categorized into various subfields like **machine learning, natural language processing, computer vision, robotics**, and more. Each of these areas focuses on different aspects of creating intelligent systems, and collectively they contribute to the development and advancement of AI technologies.

Traditional Programming (No AI)	Programming With AI
A computer program meant to solve a specific task.	A computer program that can solve generic tasks or answer generic questions
Modification in the program leads to a change in its structure.	Even a minute piece of information in the program can be modified without affecting its structure.
Modification is not quick and easy. It may lead to affecting the program adversely.	Program modification is Quick and Easy.

Goals of AI

Stuart Russell and Peter Norvig delve into four potential goals of AI

Human approach:

Systems that **think** like humans

Systems that **act** like humans

Ideal approach:

Systems that **think rationally** (Wisely)

Systems that **act rationally**

Alan Turing's definition would have fallen under the category of “**systems that act like humans.**”

To pass Turing's AI test, the computer should have the capability of **natural language processing**, it should be able to **represent and store knowledge** in machine understandable format, organize them, retrieve properly and **provide decisions** or inferences.

Importance of AI

Automation and Efficiency: AI enables automation of **repetitive tasks or routine operations** which will increase efficiency and productivity and **allows humans** to focus on **more complex and creative activities**.

Decision Making and Prediction: AI systems can **analyze vast amounts of data** to make predictions, identify patterns, and assist in decision-making. This capability aids businesses in making more choices and strategies.

Improved Healthcare: AI contributes to advancements in **medical imaging, diagnostics, personalized medicine, and drug discovery**. It helps in **early disease detection, treatment planning, and optimizing healthcare operations**.

Enhanced Customer Experiences: **AI-powered chatbots, recommendation systems, and personalization engines** improve customer interactions, providing tailored experiences and efficient support services.

Safety and Security: AI is utilized in **cybersecurity to detect threats**, identify anomalies, and protect systems from cyber attacks. It's also used in surveillance for threat detection and public safety.

Importance of AI - Continued

Innovation and Research: AI accelerates innovation by enabling researchers to **analyse complex data sets, simulate** scenarios, and explore solutions in various scientific fields.

Autonomous Systems: AI powers **autonomous vehicles, drones, and robotics**, revolutionizing transportation, logistics, and manufacturing with safer and more efficient operations.

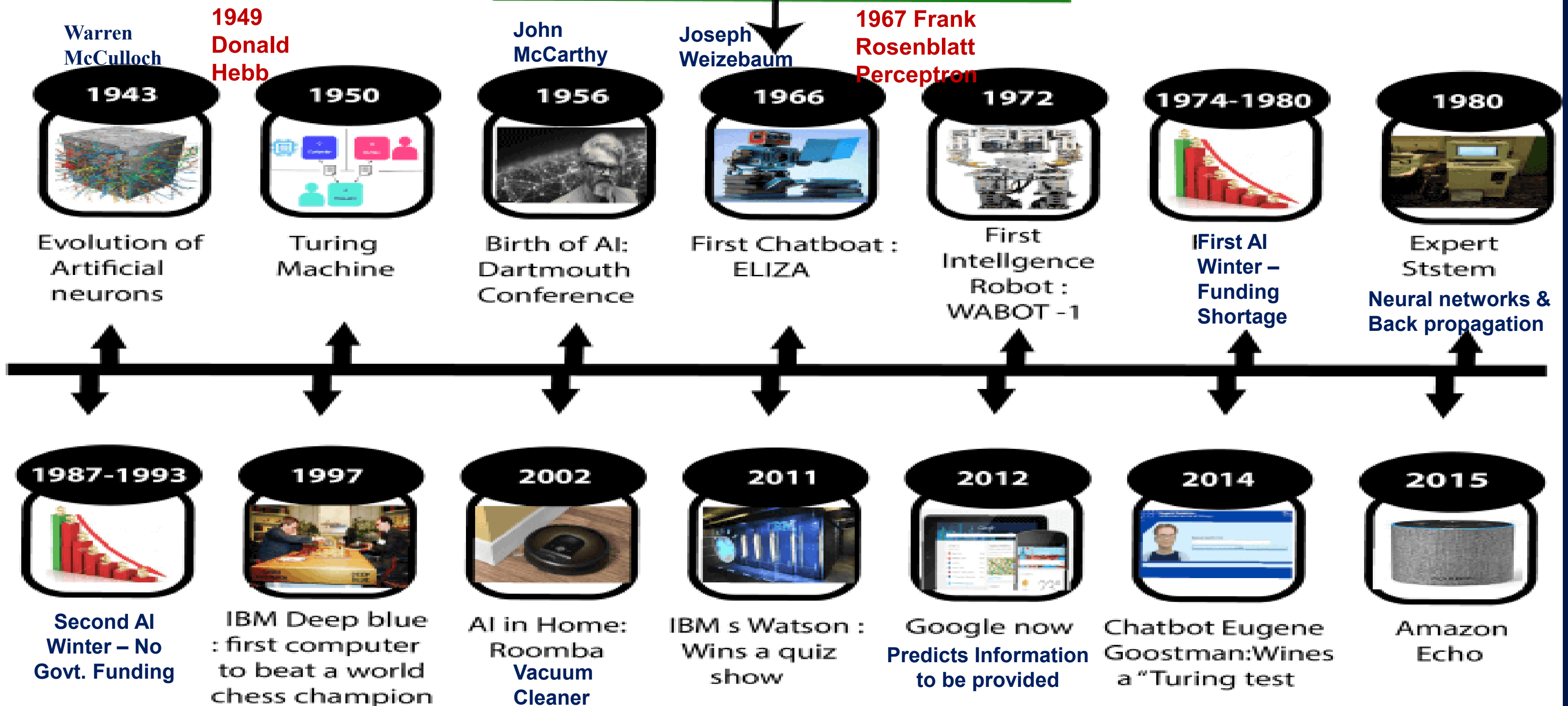
Education and Learning: AI facilitates **personalized learning** experiences, adapting to individual student needs and providing tailored educational content and feedback.

Economic Growth and Competitiveness: AI contributes to economic growth by fostering innovation, streamlining processes, and **creating new job opportunities** in emerging fields.

Ethical Considerations: Understanding and addressing ethical implications surrounding AI development, such as bias mitigation, **privacy concerns**, and responsible AI deployment, are crucial aspects of its importance.

Evolution of AI

History of AI



Evolution of AI

1943: In 1943 **Warren McCulloch** and **Walter Pits** proposed a model of artificial neurons which is now recognized as the first work in AI.

1949: **Donald Hebb** demonstrated a rule for updating connection strength between neurons which is now known as **Hebbian learning**.

1950: **Alan Turing**, an English mathematician, proposed a test to determine if a computer can exhibit the same intelligence (or the results of the same intelligence) as a human and it is called the Turing test.

1956: **John McCarthy**, an American Scientist, first coined the term 'artificial intelligence' in the first AI conference at Dartmouth College. (McCarthy would go on to invent the Lisp language.) Later that year, Allen Newell, J.C. Shaw, and Herbert Simon created the **Logic Theorist**, the first-ever running AI software program.

Evolution of AI - Continued

1966: **Joseph Weizenbaum** created the first chatbot named **ELIZA**

1967: **Frank Rosenblatt** builds the Mark 1 Perceptron, the first computer based on a neural network that 'learned' through trial and error. Just a year later, **Marvin Minsky** and Seymour Papert published a book titled **Perceptrons**, which became both the landmark work on neural networks and, at least for a while, an argument against future neural network research projects.

1972: The first intelligent humanoid robot named **WABOT-1** was built in Japan

1974-1980: During this period computer scientists had a severe shortage of funding for AI research and they called it **AI Winter**.

1980: Neural networks that use a backpropagation algorithm to train themselves become widely used in AI applications.

1980-1987: AI came back as expert systems that emulate the decision-making ability of a human expert.

1987-1993: Second AI Winter because investors and the government stopped funding AI because of the high cost involved and the results produced by AI were not efficient.

Evolution of AI - Continued

- 1997: **IBM's Deep Blue** beats the then-world chess champion **Garry Kasparov** in a chess match.
- 2002: An **AI vacuum cleaner** called **ROOMBA** entered homes as an AI application.
- 2006: Companies like Facebook, Twitter and Netflix used AI for their business.
- 2011: **IBM Watson** beats champions **Ken Jennings** and **Brad Rutter** at **Jeopardy**, a quiz show, where it had to solve complex questions as well as riddles.
- 2012: Google launched an Android app feature “**Google Now**” which was able to predict the information to be provided to the user.
- 2014: Chatbot “**Eugene Goostman**” won a competition in the Turing test.
- 2015: Baidu's Minwa supercomputer used a special kind of deep neural network called a convolutional neural network (CNN) to identify and categorize images with a higher rate of accuracy than the average human.

Evolution of AI - Continued

2016: **DeepMind's AlphaGo program**, powered by a **deep neural network**, beats **Lee Sodol**, the world champion Go player, in a five-game match. The victory is significant given the huge number of possible moves as the game progresses (over 14.5 trillion after just four moves!). Later, **Google purchased DeepMind** for USD 400 million.

2016: A social humanoid robot **Sophia** was developed by **Hanson Robotics** a Hong Kong-based company. Sophia has participated in many high-profile interviews which were covered by media around the globe. Sophia is similar to the computer program '**ELIZA**' and it also uses AI methods like **face tracking and emotion recognition** with robotic movements generated by deep neural networks. It can **emulate more than 60 facial expressions**. Its dialogue is generated by using a **decision tree**. It uses **CereProc's text-to-speech engine** to synthesize speech and this engine also allows **Sophia to sing**.

2022: A rise in large language models (**LLM**), such as **ChatGPT**, created an enormous change in the performance of AI and its potential to drive enterprise value. With these new **generative AI** practices, deep-learning models can be pre-trained on vast amounts of raw and unlabelled data.

2023: Google's Gemini (or Bard) is a generative AI chatbot developed based on LLM.

Applications of AI

Speech recognition: It is also known as automatic speech recognition (ASR), computer speech recognition, or **speech-to-text**, and it is a capability that uses natural language processing (**NLP**) to process human speech into a written format. Many mobile devices incorporate speech recognition into their systems to conduct voice searches, e.g. **Siri** a speech recognition software provides more accessibility around texting.

Customer service: **Online virtual agents** are replacing human agents along the customer journey. They answer frequently asked questions (FAQs) around topics, like shipping, or provide personalized advice, changing the way we think about customer engagement across websites and social media platforms. Examples include messaging bots on e-commerce sites with virtual agents, messaging apps, such as Slack and Facebook Messenger, and tasks usually done by virtual assistants and voice assistants.

Computer vision: This AI technology enables computers and systems to derive meaningful information from digital images, videos and other visual inputs, and based on those inputs, it can take action. This ability to provide recommendations distinguishes it from image recognition tasks.

Agriculture:

Precision Farming: AI analyzes data from drones and sensors to optimize crop yields and resource usage.

Pest Detection: Uses image recognition to identify and manage crop pests.

<https://ieeexplore.ieee.org/document/10370844>

Applications of AI - Continued

Automotive Industry:

Autonomous Vehicles: AI enables **self-driving cars** and improves safety and efficiency on roads.

Predictive Maintenance: Helps in predicting and **preventing mechanical failures** in vehicles.

Manufacturing and Industry:

Predictive Analytics: AI predicts equipment failures and **optimizes production processes** in manufacturing.

Quality Control: AI can **inspect** products **for defects** using computer vision.

Recommendation engines: Using past consumption **behavior data**, AI algorithms can help to discover data trends that can be used to develop more effective cross-selling strategies. This is used to make relevant add-on recommendations to customers during the checkout process for online retailers.

Automated stock trading: Designed to optimize stock portfolios, AI-driven high-frequency (**share**) trading platforms make thousands or even millions of trades per day without human intervention.

Medical Imaging Analysis: AI assists in diagnosing diseases from medical images like **X-rays, MRIs, and CT scans**.

Applications of AI - Continued

Finance:

Algorithmic Trading: AI-driven algorithms analyze **market trends** and execute trades autonomously.

Fraud Detection: Identifies potential fraudulent activities in **transactions or banking operations**.

Natural Language Processing (NLP) :

Chatbots and Virtual Assistants: AI-powered bots handle customer queries and provide support.

Language Translation: Translates text or speech between languages in real-time.

Entertainment:

Content Creation: AI generates content like **art, music, and stories** autonomously.

Gaming: AI powers non-player characters (NPCs) and enhances gaming experiences.

Energy:

Smart Grids: AI optimizes energy distribution and consumption in power grids.

Predictive Maintenance in Energy Systems: Helps prevent failures in energy infrastructure.

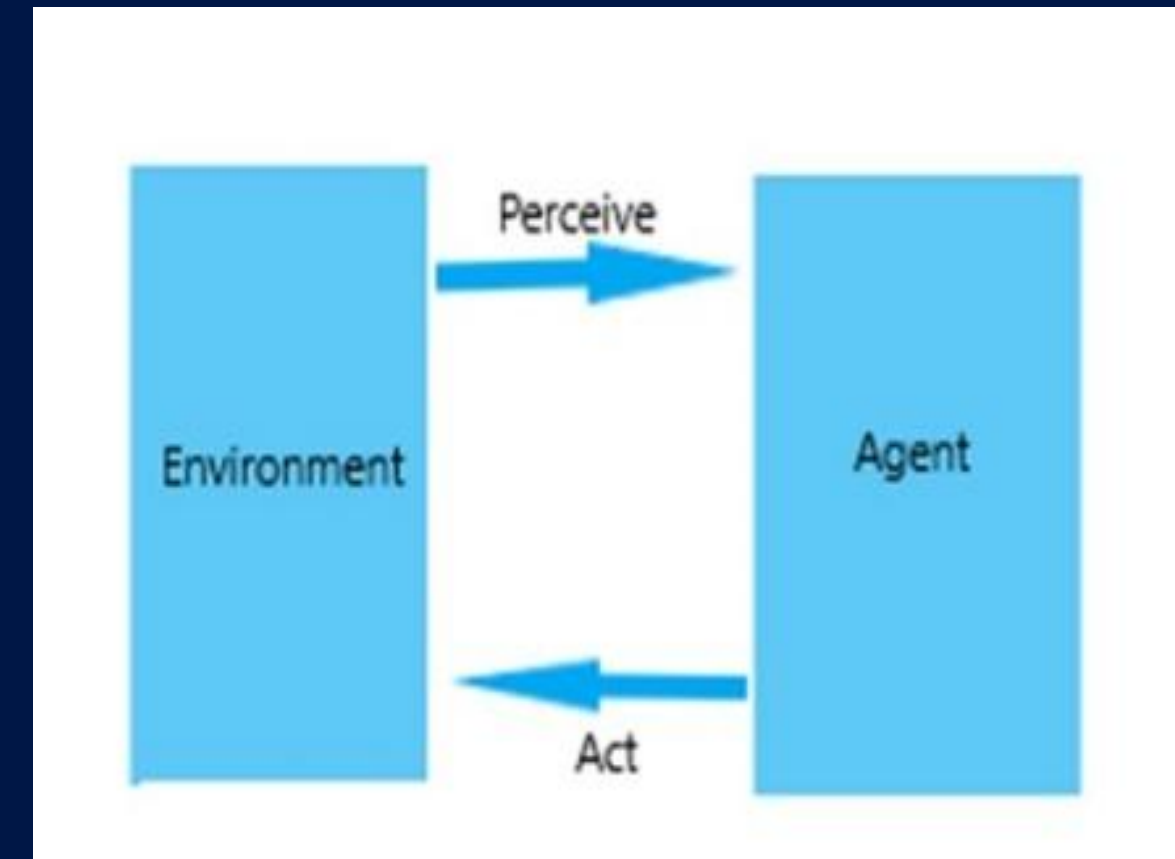
AI Systems (Agent-Environment)

In **Artificial intelligence** studies, the whole system may be divided into two components:

Agent

Environment

An agent can either perceive some information from the environment or can perform some actions on the environment.



1. Agent: The system that is responsible for acting on the environment and causing any changes to it is called an agent. An agent should be designed in such a way that it may interact with its surroundings by itself, i.e. it should not require any user input to feed information into it. It should be able to understand the situation, find the best decision that could be taken, and then act according to it.

2. Environment: Everything that surrounds the agent is the environment for the agent. It can also be defined as anything present in the universe except the agent.

Agent is an intelligent system and it can either perceive some information from the environment or perform some actions on the environment. Sensors and actuators are the main components of an agent. As per the definition of artificial intelligence, the environment, and the agent together make up the Universe. The environment can be classified based on how the agent works.

Classification of AI Systems Concerning Environment

These classifications help in understanding the nature of the environment and selecting appropriate algorithms, strategies, and approaches to tackle the challenges presented by each type of environment.

1. Accessible and Inaccessible : Although everything present around the agent is termed as environment, still the whole part may not be reached by the agent and hence cannot be accessed by the agent. Therefore, we can differentiate the environment based on whether it is accessible by the agent or not.

2. Deterministic or Non-Deterministic : If the agent is **sure** about the **perception**, then it is a deterministic environment. Deterministic environments have **clear cause-and-effect** relationships between actions and outcomes. For example, a scheduling algorithms used in logistics.

In **stochastic environments**, there is an element of randomness or **uncertainty** in the outcomes. The agent may not always be sure about the observations and perceptions it makes from the environment, in that case, the environment is non-deterministic. For example, **Probabilistic Reasoning Systems** use probability theory to model uncertainty and make decisions based on probabilistic outcomes. Applications include **weather forecasting** and risk assessment.

Classification of AI Systems Concerning Environment - Continued

3. Static or Dynamic

The environment for the agent may not always be the same as it was initially. If the environment does not change with time, then it is termed static. If there are changes in the environment with time then the environment is described as dynamic. For example, a batch Processing System processes a fixed set of data without any changes or updates once the process begins (processing historical financial data to **generate reports**). In the case of **online Systems**, they operate in real-time and continuously adapt to changing data or input (real-time monitoring and stock market analysis).

4. Discrete or continuous

If the agent can perceive and make observations from the environment continuously without any lag, then the environment is termed as continuous. If the agent observes the environment only for certain instants and the observations are not continuous, then the environment is termed as discrete.

Classification of AI Systems Concerning Environment - Continued

5. Observable or Partially-observable

In the observable environment, the agent can make observations from its complete surroundings, but in a partially observable environment, some parts of its surroundings exist but are not observable by the environment. In the case of partially observable environments **Hidden Markov Models** are used to infer unobserved data from the observed data such as in speech recognition or predicting stock prices.

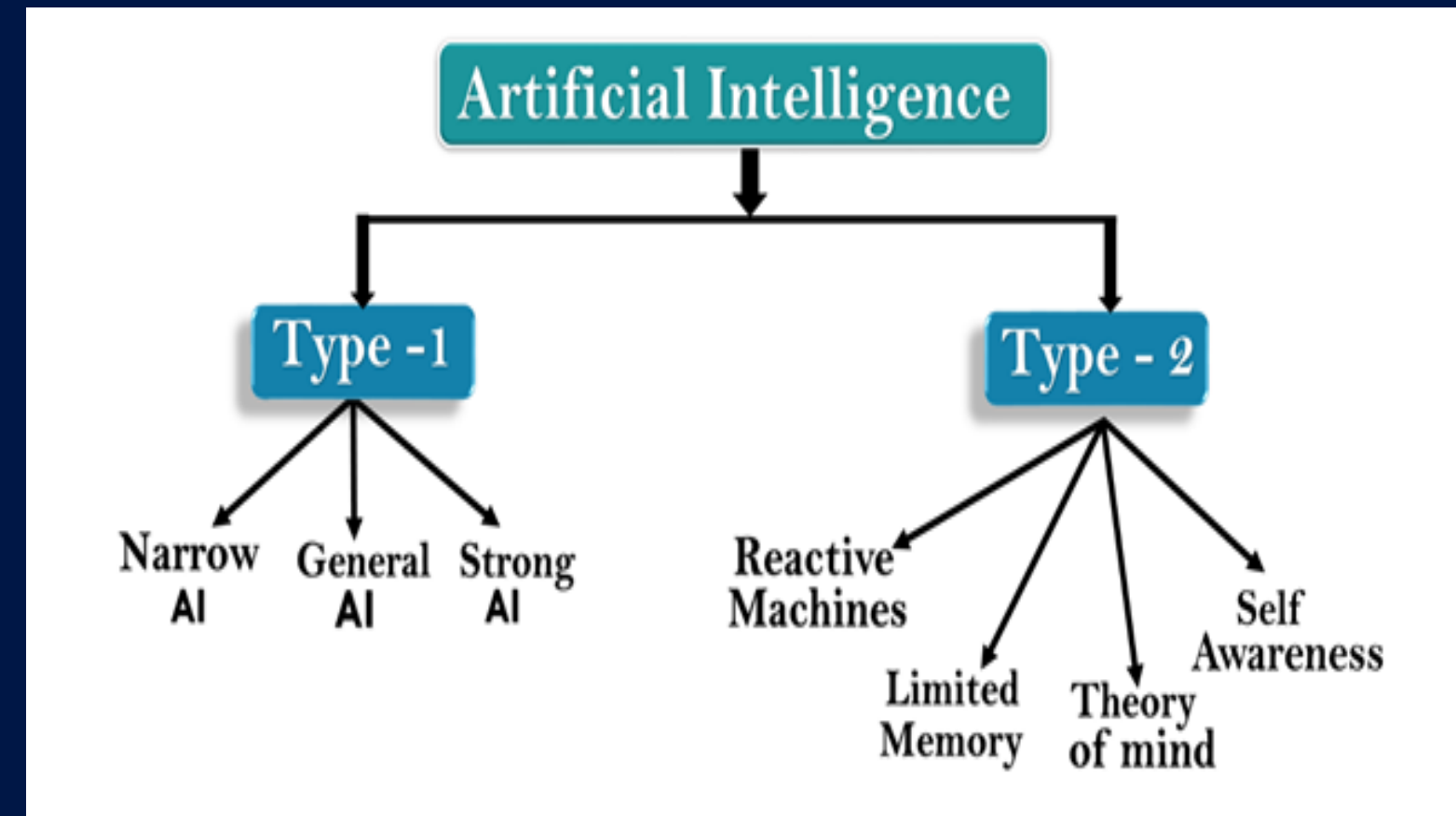
6. Adversarial Environment

Adversarial environments involve **interaction with opponents** or entities. AI systems are designed to compete against human or computer opponents in games like **chess, Go, or poker**. These systems must anticipate opponents' moves and strategize accordingly.

Types of AI Systems Based on Capabilities and Functionality

AI type-1: Based on capabilities

AI can be classified into three i) Narrow AI ii) General AI and iii) Strong AI as shown in Figure 4.



i) Weak AI or Narrow AI: (Specific field or task)

Can perform a **dedicated task** with intelligence. The most common and currently available AI is Narrow AI. Narrow AI cannot perform beyond its field or limitations, as it is only trained for one specific task. Narrow AI can fail in unpredictable ways if it goes beyond its limits.

Some Examples of Narrow AI are **playing chess**, purchasing suggestions on e-commerce sites, **self-driving cars**, **speech recognition**, and **image recognition**.

IBM's Watson supercomputer comes under Narrow AI, as it uses an **Expert system** approach combined with **Machine learning and natural language processing**.

Types of AI Systems Based on Capabilities and Functionality - Continued

ii) General AI:

General AI is a type of intelligence that can perform **any intellectual task** with efficiency like a human. They could perform smarter and **think like a human** by its own. Currently, there is no such system exists. Systems with **general AI are still under research**, and it will take lots of efforts and time to develop such systems.

iii) Super AI / Strong AI:

Super AI is a level of Intelligence of Systems at which machines could surpass human intelligence, and can perform any task better than humans with cognitive properties. It is an outcome of general AI. These AI systems should have the ability to **learn, think, reason, make judgments, plan, solve puzzles, and communicate on its own**. Super AI is **still a hypothetical concept** of Artificial Intelligence. The development of such systems in real is still a world-changing task.

Types of AI Systems Based on Capabilities and Functionality - Continued

AI type-2: Based on functionality : AI can be classified into four types based on its functionality

i) Reactive Machines

These machines only focus on **current scenarios** and react to them as per the possible best action. Such AI systems **do not store memories or past experiences for future actions**. Purely reactive machines are the most basic types of Artificial Intelligence.

IBM's Deep Blue system is an example of a reactive machines.

Google's AlphaGo is also an example of a reactive machines.

ii) Limited Memory

Limited memory machines **can store past experiences** or some data for a **short period** of time. These machines can use stored data for a limited time period only.

Self-driving Cars are one of the best examples of Limited Memory systems. These cars **can store the recent speed of nearby cars**, the **distance** of other cars, the **speed limit**, and other information to navigate the road.

Types of AI Systems Based on Capabilities and Functionality - Continued

iii) Theory of Mind

Theory of Mind AI **should understand human emotions, people, and beliefs**, and be able to **interact** socially **like humans**. This type of AI machine is still not developed, but researchers are making lots of efforts and improvements to develop such AI machines.

iv) Self-Awareness

Self-awareness AI is the future of Artificial Intelligence. These machines will be super intelligent and **will have their own consciousness, sentiments, and self-awareness**. These machines will be smarter than the human mind. Self-awareness AI **does not exist in reality** still and it is a hypothetical concept.

You can register for this course: NPTEL: **Python for Data Science**

YouTube: **Python for Data Science** Last Year Assignments -

https://www.youtube.com/playlist?list=PLuCJVhaQDfTYYYDbIpgWqm_elM2O3KmQOF

Python Data Types

1. Numeric Types:

1. **int**: Integer values (e.g., a=10, b= -5).
2. **float**: Floating-point numbers (e.g., a= 3.14, b= -0.5).
3. **complex**: Complex numbers (e.g., a=3+4j).

2. Text Type: **str**: Strings, used for text (e.g., a= "hello", b= 'world').

3. Boolean Type: **bool**: a=True , b= False, c=bool(-2) => True

4. None Type: **NoneType**: Represents the absence of a value (a=None).

5. Sequence Types (Ordered, So iterable using index)

1. **list**: Ordered, mutable collections of elements (e.g., a=[1, 2, 3]).
2. **tuple**: Ordered, immutable collections of elements (e.g., a=(1, 2, 3)).
3. **range**: Represents a sequence of numbers (e.g., a= range(5)) - Object data type

6. Set Types:

1. **set**: Unordered collection of **unique elements** i.e. no duplicate element (e.g., a={1, 2, 3})
2. **frozenset**: Immutable version of a set.

7. Mapping Type: **dict**: Key-value pairs e.g., a= {"key": "value"} , d1={"1":"A-Section", 2:"B-Section"}, Accessing the values using the key e.g. d1["1"] gives A-Section

8. Binary Types:

1. **bytes**: Immutable binary data.
2. **bytearray**: Mutable binary data.
3. **memoryview**: Memory view object for binary data.

```
l1=[1,2,3,4,5,6]
```

```
print(l1)
```

Output: [1, 2, 3, 4, 5, 6]

```
l1.insert(0,2)
```

```
print(l1)
```

Output: [2, 1, 2, 3, 4, 5, 6]

```
l2=[1, 2, 3, 4, 5,6]
```

```
l2.pop()
```

```
print(l1)
```

Output: [1, 2, 3, 4, 5]

Python Basic Libraries - NumPy

Python Basic Libraries - NumPy

NumPy:

- ❖ Numpy is a Python toolkit for working with **numeric data**, specifically with a data structure called the **Numpy array**.
- ❖ A Numpy array has a row-and-column structure
 - Numpy arrays only store numeric data
 - Numpy arrays can have more than two dimensions
 - Numpy arrays are a lot like vectors, matrices, and tensors (depending on the number of dimensions)
- ❖ Numpy also has tools for reshaping, aggregating, and cleaning the arrays.
- ❖ **Numpy has functions for:**
 - summing the values of a Numpy array
 - calculating the mean of a Numpy array (or median, or standard deviation, etc)
 - reshaping Numpy arrays
 - computing values like logarithms, exponentials, etc

NUMPY ARRAYS STORE NUMERIC DATA IN A ROW-AND-COLUMN STRUCTURE

4	12	0	4
6	11	8	4
18	14	13	7

Nested List

```
l1=[[0, 3], [10, 7], [20, 9], [30, 14], [40, 15]] #Nested list  
print(l1)
```

OUTPUT:

```
[[0, 3], [10, 7], [20, 9], [30, 14], [40, 15]]
```

#Numpy Array

```
import numpy as np  
# Create a 5x2 NumPy array.  
arr1 = np.array([[0, 3], [10, 7], [20, 9], [30, 14], [40, 15]])  
print(arr1)
```

OUTPUT:

```
[[ 0  3]  
 [10  7]  
 [20  9]  
 [30 14]  
 [40 15]]
```

arr_add=np.add(arr1,arr2)

```
[[ 1  7]
 [21 15]
 [41 19]
 [61 29]
 [81 31]]
```

arr1

```
[[ 0  3]
 [10  7]
 [20  9]
 [30 14]
 [40 15]]
```

arr2

```
[[ 1  4]
 [11  8]
 [21 10]
 [31 15]
 [41 16]]
```

arr_sub=np.subtract(arr1,arr2)

```
[[ -1 -1]
 [ -1 -1]
 [ -1 -1]
 [ -1 -1]
 [ -1 -1]]
```

arr_multiply=np.multiply(arr1,arr2)

```
[[  0  12]
 [110  56]
 [420  90]
 [930 210]
 [1640 240]]
```

arr_divide = np.divide(arr1,arr2)

```
[[0.          0.75       ]
 [0.90909091  0.875     ]
 [0.95238095  0.9       ]
 [0.96774194  0.93333333]
 [0.97560976  0.9375    ]]
```

```
arr1 = np.array([[0, 3], [10, 7], [20, 9], [30, 14], [40, 15]])
```

```
arr3=np.array([[1,2,3,4,5],[10,20,30,40,50]])
```

```
multiply= np.dot(arr1,arr3)
```

```
#Matrix Multiplication
```

```
[[ 0  3]
 [10  7]
 [20  9]
 [30 14]
 [40 15]]
```

```
[[1, 2, 3, 4, 5],
 [10,20,30,40,50]]
```

```
[[ 30  60  90 120 150]
 [ 80 160 240 320 400]
 [110 220 330 440 550]
 [170 340 510 680 850]
 [190 380 570 760 950]]
```

```
arr4 = np.array([1,2,3,4,5,6])
```

```
np.mean(arr4) # 3.5
```

```
np.std(arr4) # 1.707825127659933
```

```
np.median(arr4) # 3.5
```

```
np.reshape(arr4,(3,2))
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
np.exp(arr4)
```

```
array([ 2.71828183,  7.3890561 , 20.08553692,
        54.59815003, 148.4131591 , 403.42879349])
```

```
np.power(arr4,2)
```

```
array([ 1,  4,  9, 16, 25, 36], dtype=int32)
```

Pandas and Data Exploration

Pandas:

- ❖ Pandas is a toolkit for doing data wrangling with a special **data structure** called a **dataframe**.
- ❖ Dataframes store data (both strings and numeric) in a **row-and-column structure**.
- ❖ Data has a mix of numbers and categories
- ❖ **Pandas gives you tools to:**
 - import data from external sources (Excel Sheet)
 - create dataframes
 - format dataframes (i.e., creating or changing indexes)
 - filter data
 - aggregate data
 - reshape data
 - analyze data
 - and a lot more ...

Pandas – Data Exploration

```
data = [['Salt Lake City', 10, 120], ['Seattle', 5, 120], ['San Francisco', 5, 140], ['Seattle', 3, 100], ['Seattle', 1, 70],  
        ['San Francisco', 2, 100], ['Salt Lake City', 1, 60], ['San Francisco', 2, 110], ['Seattle', 4, 100], ['Salt Lake City', 2, 70]]
```

#data is a nested list

```
my_column_names = ['city', 'Experience', 'Salary']
```

```
df1=pd.DataFrame(data=data, columns= my_column_names)
```

#data frame consists of row index (automatically generated), coulumn name (should be given by us)

```
print(df1)
```

OUTPUT:

	<u>city</u>	Experience	Salary
0	Salt Lake City	10	120
1	Seattle	5	120
2	San Francisco	5	140
3	Seattle	3	100
4	Seattle	1	70
5	San Francisco	2	100
6	Salt Lake City	1	60
7	San Francisco	2	110
8	Seattle	4	100
9	Salt Lake City	2	70

shape of the data
rows and columns

```
df1.shape
```

Output:
(10,3)

Pandas Data Exploration - Get an overview of data

df1.head() #First five rows are displayed

```
      city  Experience  Salary
0  Salt Lake City      10     120
1      Seattle        5     120
2  San Francisco        5     140
3      Seattle        3     100
4      Seattle        1      70
```

df1.sample(5)

#Displays 5 randomly selected (samples) rows

```
      city  Experience  Salary
1      Seattle        5     120
8      Seattle        4     100
7  San Francisco        2     110
9  Salt Lake City        2      70
3      Seattle        3     100
```

df1.tail() #Last five rows are displayed

```
      city  Experience  Salary
5  San Francisco        2     100
6  Salt Lake City        1      60
7  San Francisco        2     110
8      Seattle        4     100
9  Salt Lake City        2      70
```

Pandas Data Exploration - Get an overview of data

#To display a row
`df1.loc[0]`

```
city      Salt Lake City
Experience      10
Salary      120
Name: 0, dtype: object
```

	<u>city</u>	Experience	Salary
0	Salt Lake City	10	120
1	Seattle	5	120
2	San Francisco	5	140
3	Seattle	3	100
4	Seattle	1	70
5	San Francisco	2	100
6	Salt Lake City	1	60
7	San Francisco	2	110
8	Seattle	4	100
9	Salt Lake City	2	70

#**Filtering** based on column value

#**row index values are also provided**

#Column Names are case sensitive

`df1[df1['city']=="Seattle"]` #Serch 'Seattle' in the 'city' column and return its row index

```
city  Experience  Salary
1  Seattle      5     120
3  Seattle      3     100
4  Seattle      1      70
8  Seattle      4     100
```

Data Exploration – Number of Null & Unique Values

df1.nunique()

#Displays **number of unique** values in each column. Checking duplicate values

```
city      3
Experience 6
Salary    6
```

city: The unique cities are ['Salt Lake City', 'Seattle', 'San Francisco'] → 3 unique values.

Experience: Unique values are [10, 5, 3, 1, 2, 4] → 6 unique values.

Salary: Unique values are [120, 140, 100, 70, 60, 110] → 6 unique values.

check for missing values:

df1.isnull().sum()

```
city      0
Experience 0
Salary    0
dtype: int64
```

	<u>city</u>	Experience	Salary
0	Salt Lake City	10	120
1	Seattle	5	120
2	San Francisco	5	140
3	Seattle	3	100
4	Seattle	1	70
5	San Francisco	2	100
6	Salt Lake City	1	60
7	San Francisco	2	110
8	Seattle	4	100
9	Salt Lake City	2	70

Pandas Data Exploration - Identify Column / Variable Type

df1.info()

#Gives Column Information

#	Column	Non-Null Count	Dtype
0	city	10 non-null	object
1	Experience	10 non-null	int64
2	Salary	10 non-null	int64

dtypes: int64(2), object(1)

memory usage: 372.0+ bytes

df1.dtypes

city	object
Experience	int64
Salary	int64

Pandas Data Exploration - Describe (Statistics) the Data

df.describe()

	Experience	Salary
count	10.000000	10.000000
mean	3.500000	99.000000
std	2.718251	25.582112
min	1.000000	60.000000
25%	2.000000	77.500000
50%	2.500000	100.000000
75%	4.750000	117.500000
max	10.000000	140.000000

The `df.describe()` provides the following details for each numeric column

The **count** is the number of non-null entries in each column.

mean is the mean value of all the entries in the column.

std is the standard deviation of the corresponding column.

min and max are the minimum and maximum entries in the column respectively.

25%, 50% and 75% are the First Quartile, Second Quartile(Median) and Third Quartile respectively, which gives us important info on the distribution of the dataset and makes it simpler to apply an ML model.

	<u>city</u>	Experience	Salary
0	Salt Lake City	10	120
1	Seattle	5	120
2	San Francisco	5	140
3	Seattle	3	100
4	Seattle	1	70
5	San Francisco	2	100
6	Salt Lake City	1	60
7	San Francisco	2	110
8	Seattle	4	100
9	Salt Lake City	2	70

Writing and Loading Frames in Excel

Write the Data Frame as a csv file

```
df1.to_csv("file1.csv")
```

	A	B	C	D
1	city	Experience	Salary	
2	Salt Lake City	10	120	
3	Seattle	5	120	
4	San Francisco	5	140	
5	Seattle	3	100	
6	Seattle	1	70	
7	San Francisco	2	100	
8	Salt Lake City	1	60	
9	San Francisco	2	110	
10	Seattle	4	100	
11	Salt Lake City	2	70	
12				

Read / Load the Data and identify the variables

```
df2=pd.read_csv("file1.csv")
```

#while reading , the index i.e. row number

#will be automatically included

```
print(df2.head())
```

```
      city  Experience  Salary
0  Salt Lake City      10     120
1      Seattle        5     120
2  San Francisco        5     140
3      Seattle        3     100
4      Seattle        1      70
```

Comparison Table

Feature	Nested List	NumPy Array	Pandas DataFrame
Heterogeneous Data	Yes	No	Yes
Indexing	Numerical indices	Numerical indices	Label-based indices
Ease of Use	Easy	Moderate	Advanced
Speed (Numerical Ops)	Slow	Fast	Moderate
Use Case	General-purpose	Numerical analysis	Data analysis

Matplotlib

Python Basic Libraries - Matplotlib

Matplotlib:

- ❖ used to create static, animated, and **interactive visualizations** in Python.
- ❖ you can create standard charts like **line charts, bar charts, histograms, and boxplots**.
- ❖ **Seaborn** is built on top of matplotlib itself
- ❖ With Seaborn, you can easily create density charts, hex plots, pair plots, small multiple charts, and more

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
data = {'City': ['A', 'B', 'C', 'D', 'E'],
        'Population': [500, 700, 200, 900, 300],
        'Growth': [2.5, 3.0, 1.8, 2.0, 2.2]}
df = pd.DataFrame(data)
```

- Line Chart: Best for showing trends or relationships.
- Bar Chart: Useful for comparing categories.
- Pie Chart: Shows how a total amount is broken down into various categories.
- Histogram: Good for understanding distribution.
- Boxplot: Highlights data spread and outliers.

Line Chart

Purpose: Shows trends or changes over time.

Example: Population growth in different cities.

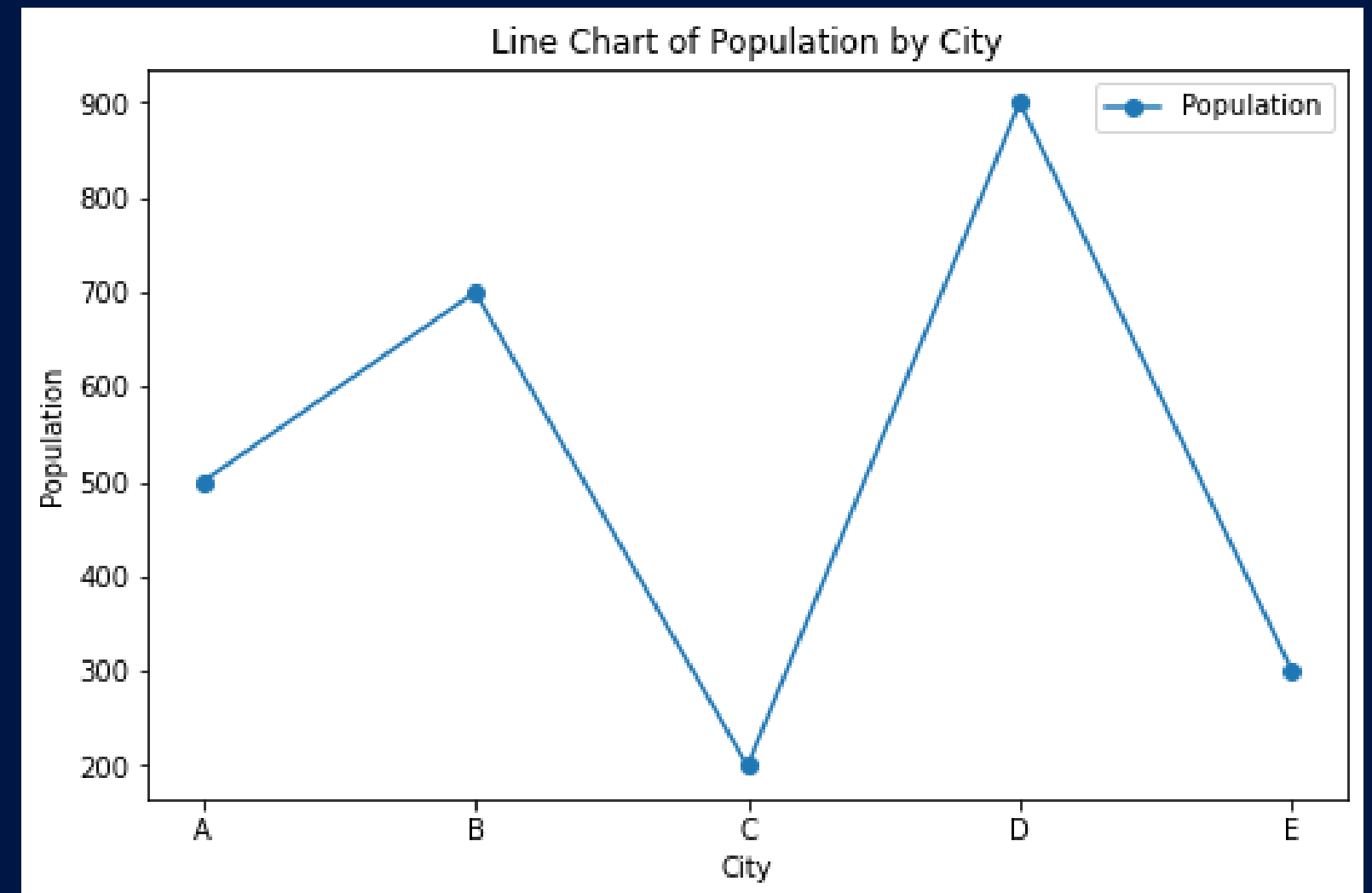
```
plt.figure(figsize=(8, 5)) # Set figure size
plt.plot(df['City'], df['Population'], marker='o', label='Population')
plt.title("Line Chart of Population by City")
plt.xlabel("City")
plt.ylabel("Population")
plt.legend()
plt.show()
```

The **label** will be displayed only when legend() is executed

Data for your reference:

'City': ['A', 'B', 'C', 'D', 'E'],

'Population': [500, 700, 200, 900, 300]



Bar Chart

Purpose: Compares categories using rectangular bars.

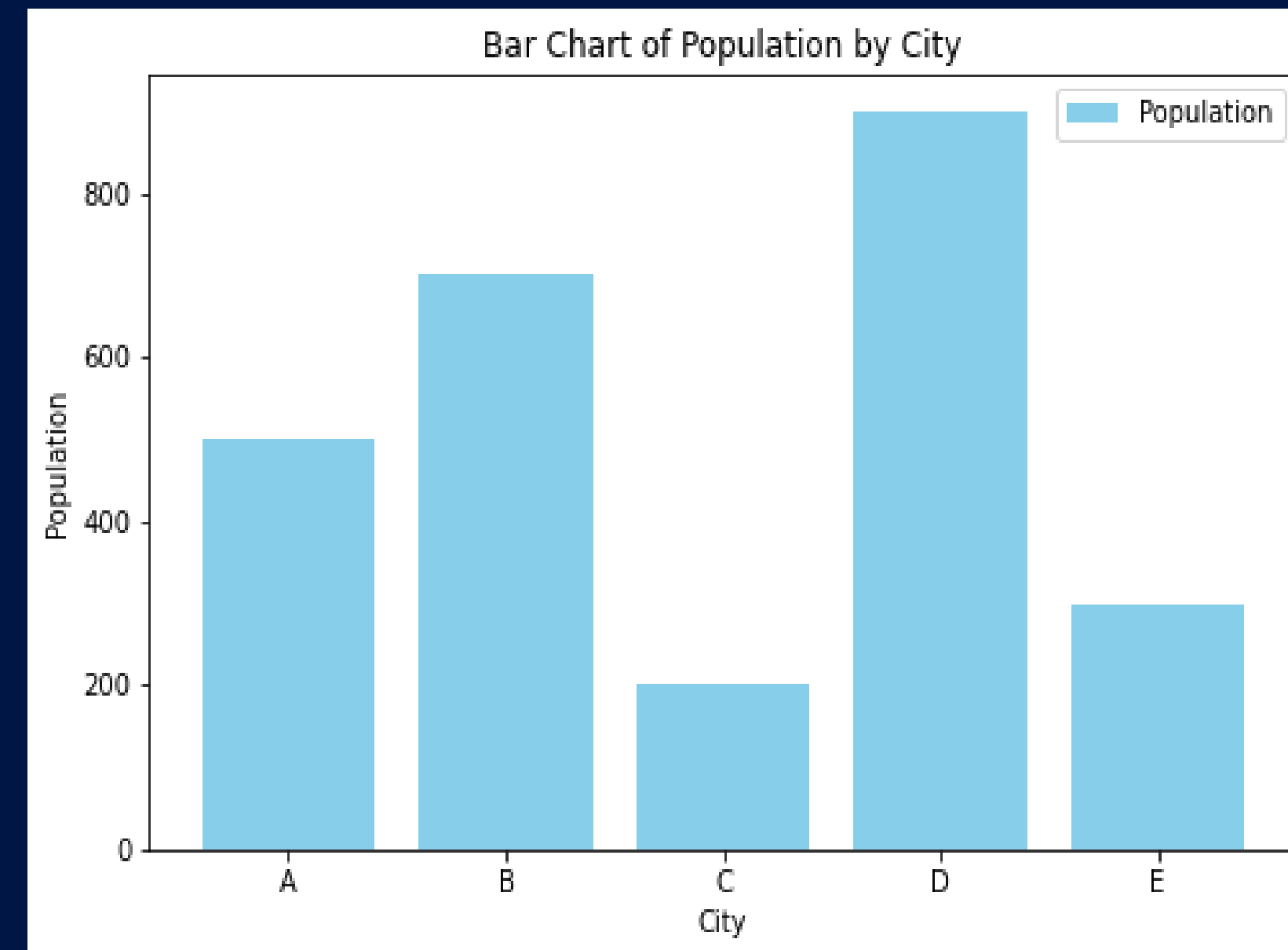
Example: Population of different cities.

```
plt.figure(figsize=(8, 5))  
plt.bar(df['City'], df['Population'], color='skyblue', label='Population')  
plt.title("Bar Chart of Population by City")  
plt.xlabel("City")  
plt.ylabel("Population")  
plt.legend() #This displays the label given in plt.bar()  
plt.show()
```

Data for your reference:

'City': ['A', 'B', 'C', 'D', 'E'],

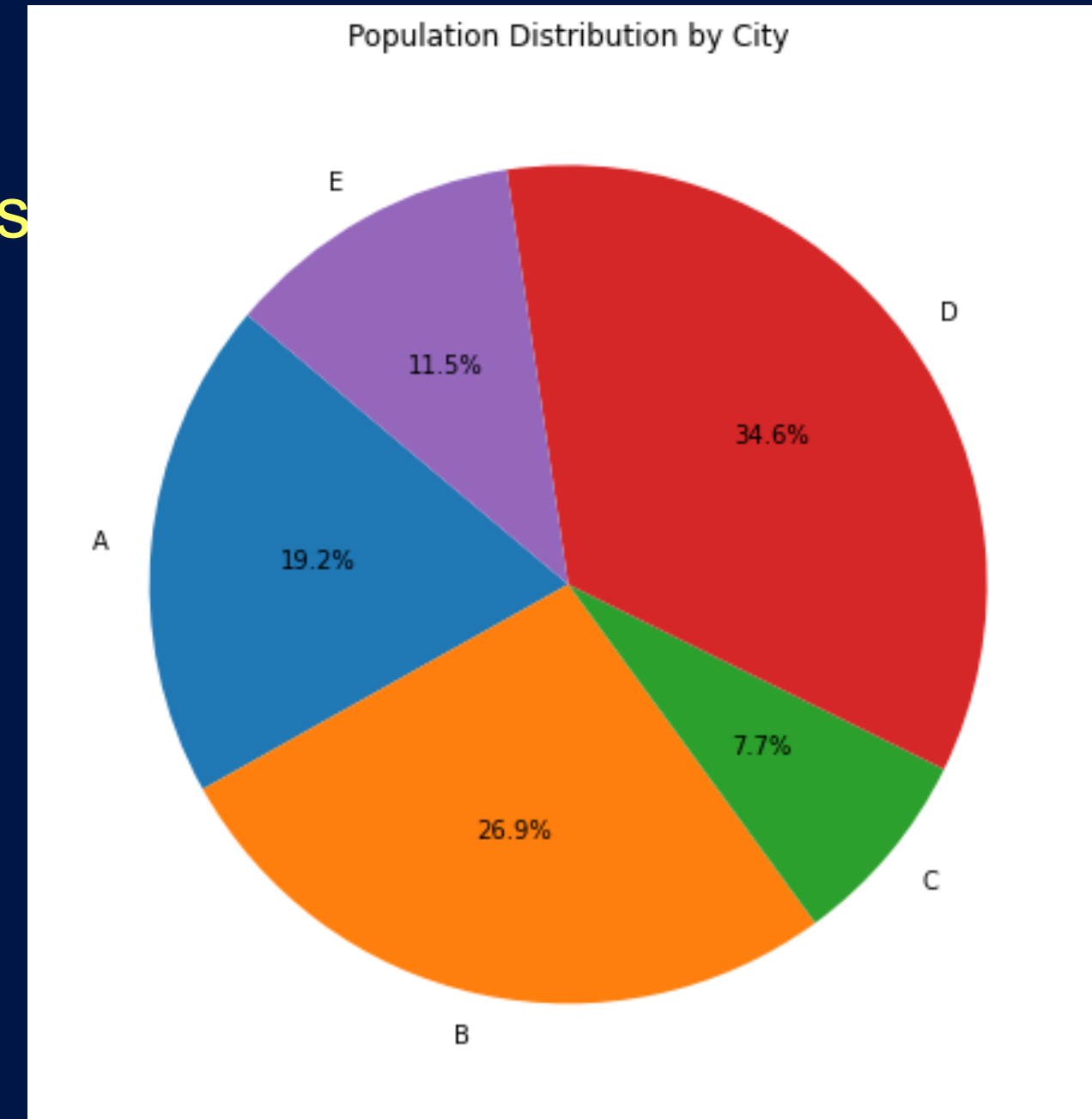
'Population': [500, 700, 200, 900, 300],



Pie Chart

```
plt.figure(figsize=(8, 8)) # Set figure size
plt.pie(df['Population'], labels=df['City'], autopct='%1.1f%%', startangle=140)
#%1.1f: Formats the number as a float with 1 decimal place.
#%: Adds a literal percent sign to the end. (e.g., 12.5%).
#startangle=140: Starts the first slice at an angle of 140 degrees
#By default startangle 0 deg means +x axis
#startangle 90 deg means +y axis (rotates anti-clockwise)
# Adding a title
plt.title("Population Distribution by City")
# Display the chart
plt.show()
```

Percentage of A = $\{\text{City Population} / \text{Total Population}\} \times 100$
= $\{500 / (500 + 700 + 200 + 900 + 300)\} \times 100$
= $\{500 / 2600\} \times 100$
= 19.2%



'City': ['A', 'B', 'C', 'D', 'E'],
'Population': [500, 700, 200, 900, 300]

Histogram

Purpose: Displays the frequency distribution of numerical data.

Example: Distribution of population values.

```
plt.figure(figsize=(8, 5))  
plt.hist(df['Population'], bins=3, color='orange', edgecolor='black')  
plt.title("Histogram of Population")  
plt.xlabel("Population Range")  
plt.ylabel("Frequency")  
plt.show()
```

Explanation:

'Population': [500, 700, 200, 900, 300]

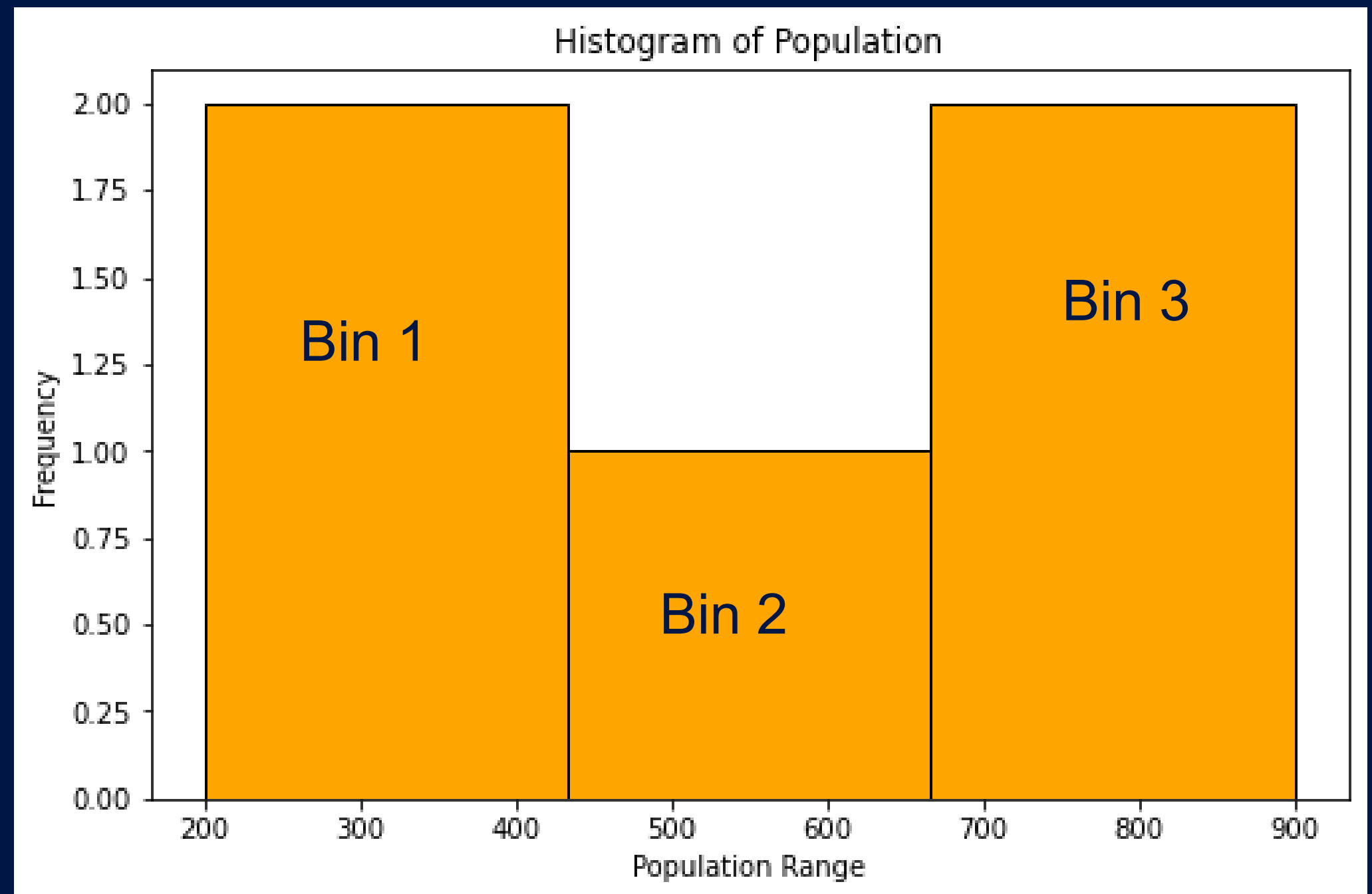
Width of each bin = $(\text{max}-\text{min})/\text{no.of bins}$
 $= (900-200)/3 = 233.3$

Range of 1st bin = $200+233.3=433.3$

Frequency of 1st bin (data 200,300) = 2

Range of 2nd bin = $433.3 + 233.3 = 666.6$

Frequency of 2nd bin (only data 500) = 1



Boxplot

Purpose: Summarizes the distribution of a dataset using minimum, 25th percentile, median, 75th percentile, and maximum.

Example: Population growth in cities

```
plt.figure(figsize=(8, 5))  
plt.boxplot(df['Growth'], vert=False, patch_artist=True)  
plt.title("Boxplot of Growth Rate")  
plt.xlabel("Growth Rate")  
plt.show()
```

'Growth': [2.5, 3.0, 1.8, 2.0, 2.2]

Arrange in Ascending order = [1.8,2.0,2.2,2.5.,3.0]

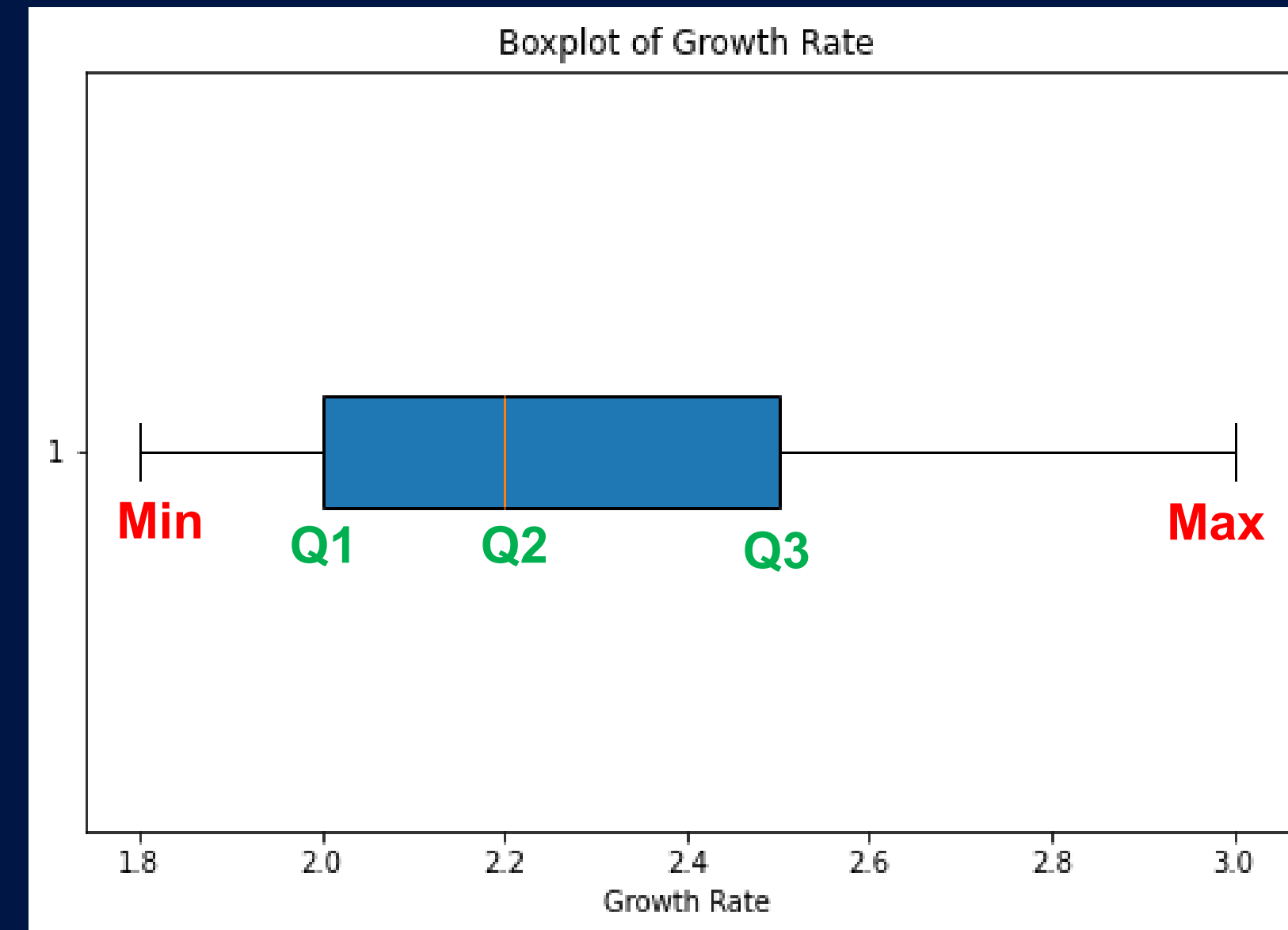
Min = 1.8

Quartile 2 = Q2 = Median = Middle value =2.2

Lower Quartile = Q1= Median of the lower half =2.0

Upper Quartile = Q3= Median of the upper half = 2.5

Max = 3.0



Anonymous or Lambda Single Line Functions

Sometimes we need to declare a function without any name. The nameless property function is called an anonymous function or lambda function.

Syntax: `lambda argument: conditionexpression`

Example 1

```
fun = lambda x : x % 2 == 0  
print(fun(2)) # True actual argument is passed while calling the function  
print(fun(3)) # False
```

To pass the actual value to the dummy argument x, assign the lambda function to a variable. Now this variable becomes the function name and pass the value for x as an argument to this function.

Lambda functions are more useful when we pass a function as an argument to another function. We can also use the lambda function with built-in functions such as filter, map, reduce because this function requires another function as an argument.

Single Line Code (Anonymous or Lambda Functions)

Example 2: Lambda function with three arguments

```
x=lambda a,b,c:a+b+c  
print(x(5,6,2))
```

OUTPUT:

13

Example 2: Lambda function with in another function.

```
def myfunc(n):  
    return lambda a:a*n  
mydoubler = myfunc(2) # mydoubler has the lambda function with n value and argument a  
                    # but only 'n' value is passed as parameter here.  
print(mydoubler(11)) # argument a is passed
```

OUTPUT:

22

Lambda Functions Passed to map() Function

Example 4:

for every element present in the sequence, perform a cube operation and generate a new cube list.

```
list1 = [2, 3, 4, 8, 9]
list2 = list(map(lambda x: x*x*x, list1))
#for every x value in the list1 compute x*x*x
print("Cube values are:", list2)

#One line map function
print(list(map(lambda a: a + 2, [5, 6, 7, 8, 9, 10])))

# [7, 8, 9, 10, 11, 12]
```

Output:

Cube values are: [8, 27, 64, 512, 729]

Map function applies the lambda function to each element of list1. It returns an iterable containing the cube of each element.

Filter does not alter the value. But map function maps original value to a different value

Lambda Function Passed to filter() Function

Example 5:

Program to print even numbers from a list with a lambda function

```
list1 = [10, 5, 12, 78, 6, 1, 7, 9]
even_nos = list(filter(lambda x: x % 2 == 0, list1))
print("Even numbers are: ", even_nos)
```

Output: Even numbers are: [10, 12, 78, 6]

The filter function accepts lamda function as one argument and a list as another argument.

The filter function passes the values from the list one by one to the lambda function and get it checked

If the expression is True then the filter() function returns an iterator (index & value) containing elements from the list 1.

By converting this iterator to a list using list(), we obtain a list of even numbers which is stored in the variable even_nos.

Lambda Function Passed to reduce() Function

In Python, the reduce() function is used to minimize sequence elements into a single value by applying the specified condition.

The reduce() function is present in the **functools module**; hence, we need to import it using the import statement before using it.

Example 6:

```
from functools import reduce
```

```
list1 = [20, 13, 4, 8, 9]
```

```
add = reduce(lambda x, y: x+y, list1)
```

```
# Initially the first element will be in x, and every y value in the list1 starting from the second value, will be added to x until the end of the list1. The result will be returned
```

```
print("Addition of all list elements is : ", add)
```

Output:

Addition of all list elements is: 54

The reduce function effectively reduces the iterable (in this case, list1) to **a single accumulated value**.

Lambda Function with filter() Function and all() Function

Find primes in a range in a single line of code

```
print(list(filter(lambda a: all(a % b != 0 for b in range(2, a)), range(2,20))))
```

```
# [2, 3, 5, 7, 11, 13, 17, 19]
```

range 2,20 provides 2 to 19, each element is passed as argument 'a' to lambda.

b is from 2 to a. if $a \% b \neq 0$ then it is not a prime number

'all' checks the if condition from 2 to a

Class & Object

One-line class

#Regular way

```
class Emp:
```

```
    def __init__(self, name, age): #constructor method
```

```
        self.name = name
```

```
        self.age = age
```

```
emp1 = Emp("a44", 22)
```

```
print(emp1.name, emp1.age) # Output: a44 22
```

#Single line way

#Method 1 Lambda with Dynamic Attributes

```
Emp = lambda: None; Emp.name = "a44"; Emp.age = 22 #Emp is object like emp1
```

```
print(Emp.name, Emp.age)
```

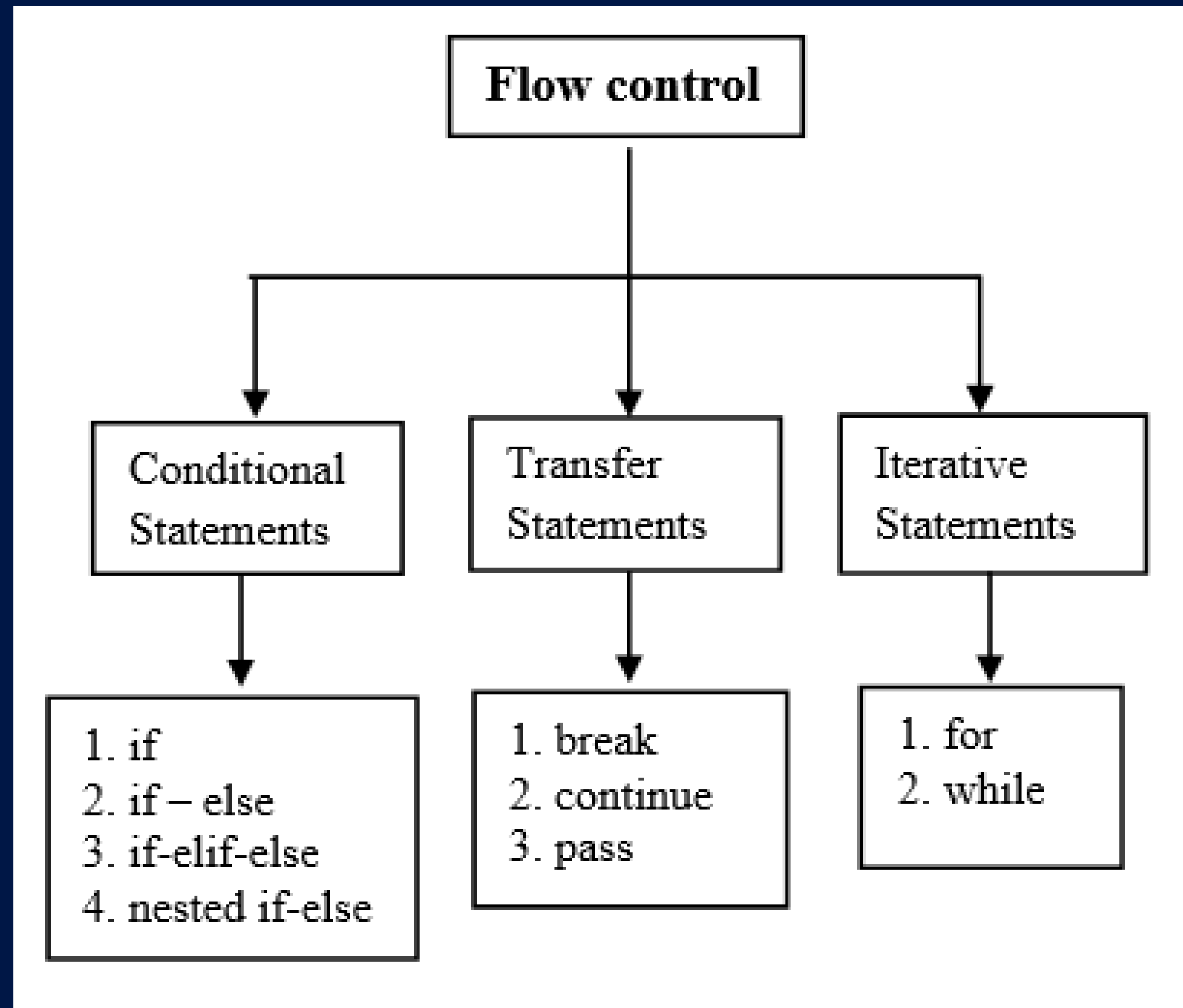
#Method 2

```
from collections import namedtuple
```

```
Emp = namedtuple('Emp', ["name", "age"]) ("a44", 22)
```

```
print(Emp.name, Emp.age)
```

Iterative / Looping Statements



Program Control Structure - if

i) The if statement

```
x = 0
```

```
y = 5
```

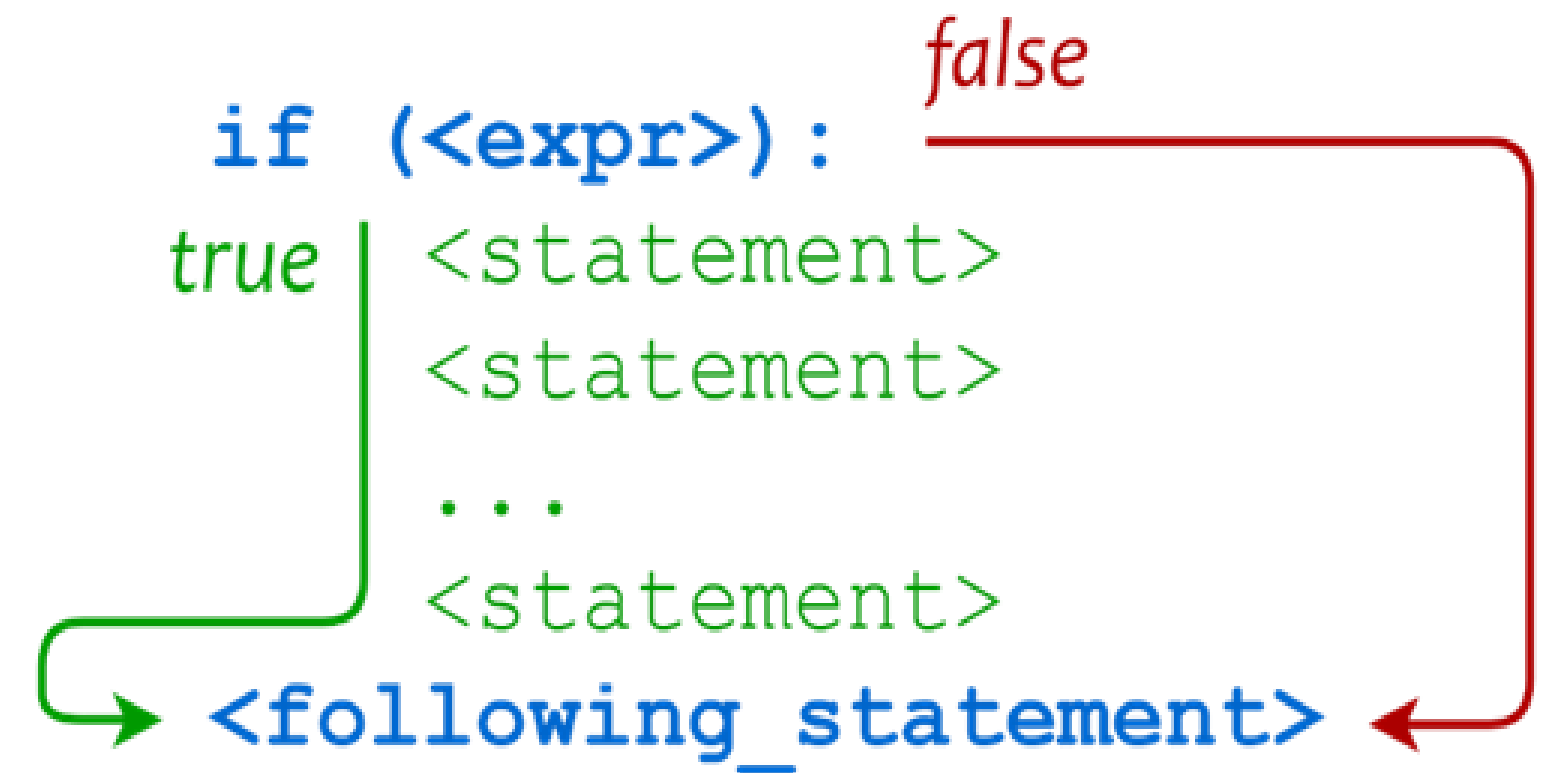
```
if x < y:
```

```
    print('yes')
```

True

Output:

Yes



Program Control Structure – if ... else

Example 6

```
if 'foo' in ['bar', 'baz', 'qux']:
    print('Expression was true')
    print('Executing statement in suite')
    print('...')
    print('Done.')
print('After conditional')
```

Output: After conditional

Example 7

```
x = 20
if x < 50:
    print('x is small')
else:
    print('x is large')
```

Output:
x is small

Program Control Structure – if ... elif ... else

Example 8

```
name = 'Joe'
if name == 'Fred':
    print('Hello Fred')
elif name == 'Xander':
    print('Hello Xander')
elif name == 'Joe':
    print('Hello Joe')
elif name == 'Arnold':
    print('Hello Arnold')
else:
    print("I don't know who you are!")
```

Output:
Hello Joe

<expr1> if <conditional_expr> else <expr2>

```
if True:
    print('foo')    #you will get indentation error
```

To avoid this you can use pass.

```
if True:
    pass
```

```
print('foo')
```

Iterative / Looping Statement - while

Example 9

```
count = 1
# run loop till count is less than 5
while count < 5:
    print(count)
    count = count + 1
```

Output:

1
2
3
4

Example 8

Check how many times a given number can be divided by 3 before it is less than or equal to 10.

```
count = 0
number = 180
while number > 10:
    # Divide number by 3
    number = number / 3
    # increase count
    count = count + 1
print('Total iteration required', count)
```

Output:

Total iteration required 3

Iterative / Looping Statement - while

Example 10(a)

```
n = int(input('Please Enter Number '))
while n > 0:
    # check even and odd
    if n % 2 == 0:
        print(n, 'is an even number')
    else:
        print(n, 'is an odd number')
    # Decrease the number by 1 in each iteration
    n = n - 1
```

output:

```
Please Enter Number 7
7 is an odd number
6 is an even number
5 is an odd number
4 is an even number
3 is an odd number
2 is an even number
1 is an odd number
```

Iterative / Looping Statement – while ... continue ... break

Example 10(b):

n=10

while n > 0:

check even and odd

if n==5:

n=n-1

continue

check even and odd

if n % 2 == 0:

print(n, 'is an even number')

else:

print(n, 'is an odd number')

Decrease the number by 1 in each iteration

n = n - 1

when n=5 the
while loop goes
to next iteration
and continues

OUTPUT:

10 is an even number
9 is an odd number
8 is an even number
7 is an odd number
6 is an even number
4 is an even number
3 is an odd number
2 is an even number
1 is an odd number

Example 10(c):

n=10

while n > 0:

if n==5:

break

check even and odd

if n % 2 == 0:

print(n, 'is an even number')

else:

print(n, 'is an odd number')

Decrease the number by 1 in each iteration

n = n - 1

while loop is broken
when n=5

Output:

10 is an even number
9 is an odd number
8 is an even number
7 is an odd number
6 is an even number

Iterative / Looping Statement - while

Example 11: Use nested while loop to print pattern

*

* *

* * *

* * * *

i = 1

outer while loop

4 rows in pattern

while i < 5:

 j = 0

 # nested while loop

 while j < i:

 print('*', end=' ')

 j = j + 1

 # end of nested while loop

 # new line after each row

 print("")

 i = i + 1

Using for loop inside the while loop

i = 1

outer while loop

while i < 5:

 # nested for loop

 for j in range(1, i + 1):

 print("*", end=" ")

 print("")

 i = i + 1

Iterative / Looping Statements - for with tuple & range ()

Example : tuple

```
t1=(10,15,20,25)
for i in t1:
    print(i)
```

Output:

10
15
20
25

Example : range()

```
for i in range(2,5):
    print(i)
```

Output:

2
3
4

Example : range()

```
for i in range(0,10,2):
    print(i)
```

Output:

0
2
4
6
8

Iterative / Looping Statement - for

Reverse the numbers in the given list:

Example 12(a):

using index

```
numbers = [1, 2, 3, 4]
for i in numbers[::-1]:
    #start index 3 , end index -1, step -1
    print(i)
```

Output:

4
3
2
1

Example 12(b):

Using reversed() function

```
list1 = [10, 20, 30, 40]
for num in reversed(list1):
    print(num)
```

Output:

40
30
20
10

Example 12(c):

Using range function

```
num = 5
# start = 5
# stop = -1
# step = -1
for num in (range(num, -1, -1)):
    print(num)
```

Output:

5
4
3
2
1
0

Iterative / Looping Statements - for with enumerated list

Example 13: Print elements of the list with its index number using the enumerate() function

The enumerate function returns the index of a list element following its value. It returns index and value for all elements one after another.

```
list1 = [4, 2, 5, 7, 8]
for i, v in enumerate(list1):
    print("list[" + str(i) + "] = ", v)
```

Output:

```
list[ 0 ] = 4
list[ 1 ] = 2
list[ 2 ] = 5
list[ 3 ] = 7
list[ 4 ] = 8
```

Example 14:

```
for i, v in enumerate([15,25,35,45,55]):
    print(i, " ", v)
```

Output:

```
0  15
1  25
2  35
3  45
4  55
```

Iterative / Looping Statements - for with enumerated list

Example :

```
fold_names_list = ["fold1", "fold2", "fold3", "fold4", "fold5"]
```

```
for index, item in enumerate(fold_names_list , start=1): #index value is changed  
    print(index, item, "\n")
```

fold_num	split_name
1	"fold1"
2	"fold2"
3	"fold3"
4	"fold4"
5	"fold5"

Iterative / Looping Statements - for with list comprehension

Example 15: list comprehension.

Bad Practice if used only for printing

```
list1 = [1, 2, 3, 7, 8]
# list comprehension
[print(i) for i in list1]
```

Output:

1
2
3
7
8

Example 16: List comprehension

```
list1 = [1, 2, 3, 7, 8]
squares = [x**2 for x in list1]
print(squares)
# Output: [1, 4, 9, 49, 64]
```

Example 17: List comprehension

```
print([i + 1 for i in [1, 3, 6, 10] if i % 2 == 1])
```

Output: [2, 4]

Python Programming

Example 19:

Analyse the following code and decide the data type and the value of the variable name 'a' that may be obtained by a Python Interpreter. K4

```
a=-9//8.9
```

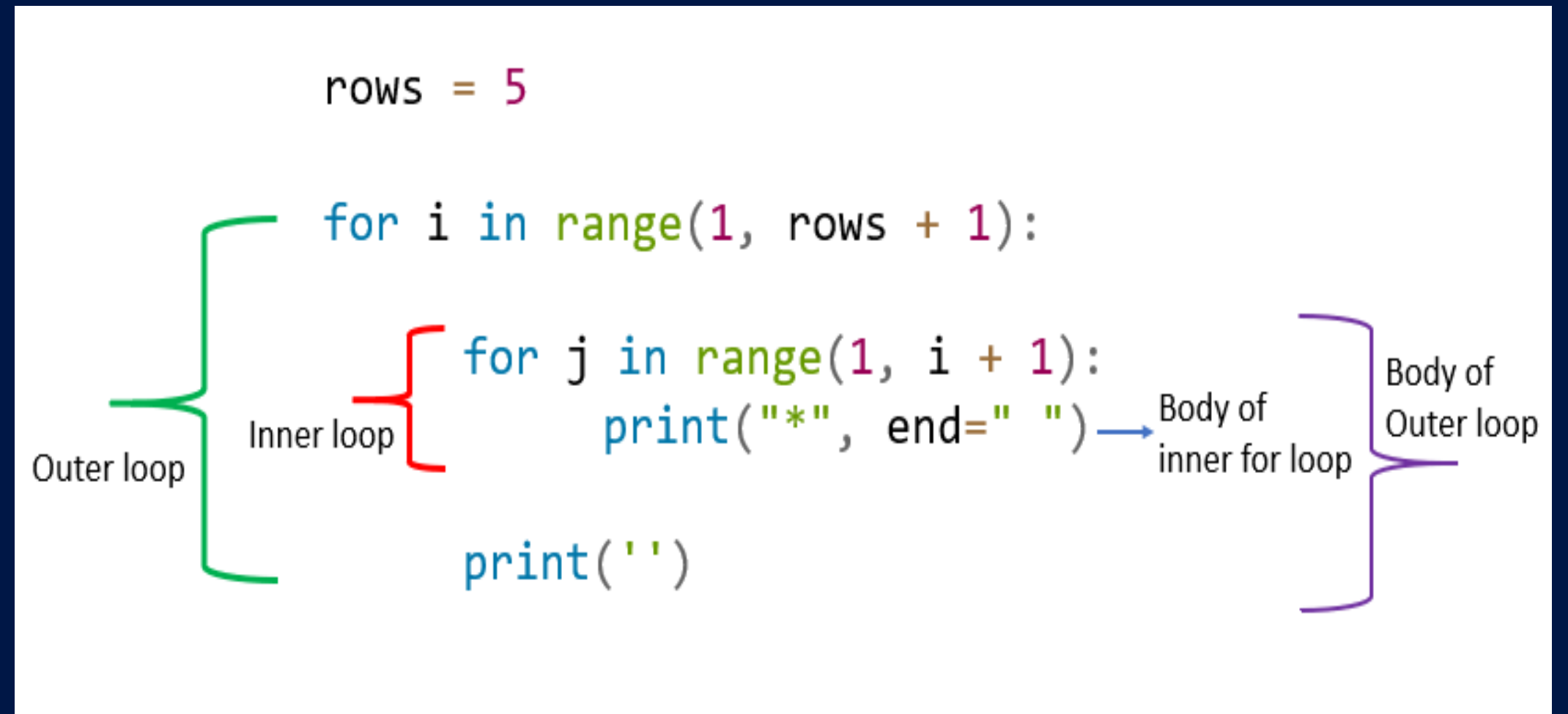
Ans: Here // indicates floor division. When both the numbers are integers, the result is an integer number. If any one number is float the result is a float number. So, here 'a' is a float number. The result of the division will be between -1 and -2. In the floor division, **the result will be the nearest lower integer**. So here, the lower integer is -2 not -1. So 'a' will be -2 but a float number i.e. 'a' = -2.0

Iterative / Looping Statements - nested for with range()

Example 18(d): Using nested for loop print the following pattern

```
*  
* *  
* * *  
* * * *  
* * * * *
```

```
rows = 5  
# outer loop  
for i in range(1, rows + 1):  
    # inner loop  
    for j in range(1, i + 1):  
        print("*", end=" ")  
        #end=" " prevents new line after each star  
    print("")  
#empty print statement moves the cursor to next line
```



Functions

Python supports two types of functions.

1. Built-in function
2. User-defined function

The functions that come along with Python itself are called built-in functions or predefined functions. Some of them are listed below.

`range()`, `id()`, `type()`, `input()`, `eval()` etc.

Functions that are created by the programmer explicitly according to the requirement are called user-defined functions.

User Defined Functions

The diagram illustrates the components of a Python function definition. It shows a function named `add` that takes two parameters, `num1` and `num2`. The function body contains three lines of code: `print("Number 1:", num1)`, `print("Number 2:", num1)`, and `addition = num1 + num2`. The `return` statement `return addition` is annotated as the 'Return Value'. Below the function definition, a function call `res = add(2, 4)` is shown, followed by `print(res)`. The function call is annotated as 'Function call'.

```
def add(num1, num2):  
    print("Number 1:", num1)  
    print("Number 2:", num1)  
    addition = num1 + num2  
  
    return addition  
  
res = add(2, 4)  
print(res)
```

Annotations in the diagram:

- Function Name:** `add`
- Parameters:** `num1, num2`
- Function Body:** The block of code between `def` and `return`.
- Return Value:** `addition`
- Function call:** `add(2, 4)`

```
# function  
def message():  
    print("Welcome to PYnative")  
# call function using its name  
message()
```

Output:
Welcome to PYnative

User Defined Functions

function with parameters

```
def course_func(name, course_name):  
    print("Hello", name, "Welcome to III ECE A")  
    print("Your course name is", course_name)
```

call function

```
course_func('John', 'AI and ML')
```

Output:

```
Hello John Welcome to III ECE A  
Your course name is AI and ML
```

function with parameters and return value

```
def calculator(a, b):  
    add = a + b  
    # return the addition  
    return add
```

call function

take the return value in the variable

```
res = calculator(20, 5)  
# function is called with parameters passed  
print("Addition :", res)
```

Output:

```
Addition: 25
```

User Defined Functions – Positional Arguments

Python Function Arguments

1. Positional arguments
2. keyword arguments
3. Default arguments
4. Variable-length arguments

```
def add(a, b):  
    print(a - b)
```

```
add(50, 10)
```

```
add(10, 50)
```

```
add(105, 561, 4)
```

Output:

40

-40

Type Error: add() takes 2 positional arguments but 3 were given

NB: In the positional argument number and position of arguments must be matched. If we change the order, then the result may change. Also, If we change the number of arguments, then we will get an error.

User Defined Functions – Keyword Arguments

```
def message(name, surname):  
    print("Hello", name, surname)
```

```
message(name="John", surname="Wilson")  
message(surname="Wilson", name="John")
```

Output:

```
Hello John Wilson  
Hello John Wilson
```

A keyword argument is an argument value, passed to function preceded by the variable name and an equals sign.

In keyword arguments order of argument is not matter, but the number of arguments must match. Otherwise, we will get an error.

User Defined Functions – Positional & Keyword Arguments Together

While using keyword and positional arguments simultaneously, we need to pass **1st arguments as positional arguments** and then keyword arguments. Otherwise, we will get `SyntaxError`. See the following example. Positional or keyword? This can only be identified while calling the function.

```
def message(first_nm, last_nm):  
    print("Hello..!", first_nm, last_nm)
```

```
message("John", "Wilson") # correct use
```

```
message("John", last_nm="Wilson") # correct use
```

```
message(first_nm="John", "Wilson")
```

```
# provides SyntaxError: because
```

```
# positional argument follows keyword argument
```

User Defined Functions – Default Arguments

```
# function with default argument  
def message(name="Guest"):  
    print("Hello", name)
```

```
# calling function with argument  
message("John")
```

```
# calling function without argument  
message()
```

Output:
Hello John
Hello Guest

User Defined Functions – Variable Length Arguments

We can declare a variable-length argument with the * (asterisk) symbol

```
def add(*numbers):  
    total = 0  
    for i in numbers:  
        total = total + i  
    print("Sum is:", total)
```

0 arguments

add()

5 arguments

add(10, 5, 2, 5, 4)

3 arguments

add(78, 7, 2.5)

Output:

Sum is: 0

Sum is: 26

Sum is: 87.5

Functions – Pass by Object Reference

Python's argument-passing model is neither "Pass by Value" nor "Pass by Reference" but it is "Pass by Object Reference".

If you pass an immutable object (like an integer, string, or tuple), the function gets a reference to the object, but you cannot modify the original object. If you try to reassign it, a new object will be created, which behaves like pass by value. Python does not use pass by value in the way languages like C or Java (for primitive types) do.

Python does not use a pure pass-by-value mechanism for mutable objects like lists. Instead, Python uses a pass-by-object-reference model. If you pass a mutable object (like a list, dictionary, or set), the function gets a reference to the same object, and changes to the object inside the function will be reflected outside the function. This behaves like pass by reference.

Functions - Recursive

```
def factorial(no):  
    if no == 0:  
        return 1  
    else:  
        return no * factorial(no - 1)  
  
print("factorial is:", factorial(5))
```

Output:
factorial of a number is: 120

Explanation: The same function is called repeatedly in itself.

$5 * \text{factorial}(4)$

$5 * 4 * \text{factorial}(3)$

$5 * 4 * 3 * \text{factorial}(2)$

$5 * 4 * 3 * 2 * \text{factorial}(1)$

$5 * 4 * 3 * 2 * 1 = 120$

Single Line Code

Merging dictionaries in one line

```
d1 = { 'A': 1, 'B': 2 }
```

```
d2 = { 'C': 3, 'D': 4 }
```

#Method1

```
d1.update(d2)
```

```
print(d1) # {'A': 1, 'B': 2, 'C': 3, 'D': 4}
```

#Method2

```
d3 = {**d1, **d2}
```

```
print(d3) # {'A': 1, 'B': 2, 'C': 3, 'D': 4}
```

NB: ** unpacks the dictionary

Exception handling in one line

#Original method

```
try:
```

```
print(x)
```

```
except:
```

```
print("Error")
```

#Single line way

```
exec('try:print(x) \nexcept:print("Error")') # Error
```

Single Line Code - Continued

```
#Swap values in one line
#Single-line method
v1 = 100
v2 = 200
temp = v1
v1 = v2
v2 = temp
print(v1, v2) # 200 100
```

```
# One-line value swapping
v1, v2 = v2, v1
print(v1, v2) # 200 100
```

```
# One-line semicolon
# example 1
a = "Python"; b = "Programming"; c = "languages";
print(a, b, c)
# print
# Python Programming languages
```

Single Line Code - Continued

Single Line Code - Sorting

```
# Sort in one line  
mylist = [32, 22, 11, 4, 6, 8, 12]  
# Method1  
mylist.sort()  
print(mylist) # # [4, 6, 8, 11, 12, 22, 32]
```

#The **sort()** method sorts the list in place, meaning the original list (l1) is modified.

```
print(sorted(mylist)) # [4, 6, 8, 11, 12, 22, 32]
```

The **sorted()** function creates a new sorted list while leaving the original list unchanged.

Single Line Code

Dictionary in one line

```
mydict = ["John", "Peter", "Mathew", "Tom"]  
mydict = dict(enumerate(mydict))  
print(mydict)  
# {0: 'John', 1: 'Peter', 2: 'Mathew', 3: 'Tom'}
```

Deleting the Multiple elements from the list

```
mylist = [100, 200, 300, 400, 500]  
del mylist[1::2]  
print(mylist) # [100, 300, 500]
```

#Multiple variable assignments in one line.

#Single-line method

```
x = 5
```

```
y = 7
```

```
z = 10
```

```
print(x , y, z) # 5 7 10
```

#Single line way

```
a, b, c = 5, 7, 10
```

```
print(a, b, c) # 5 7 10
```

Single Line Code (if elif) - Continued

#if Else In a single line.

#Example 1 if else

```
print("Yes") if 8 > 9 else print("No") # No
```

#Example 2 if elif else

```
E = 2  
print("High") if E == 5 else print("STUDIO") if E == 2 else print("Low")    # STUDIO
```

#Example 3 only if

```
if 3 > 2: print("Exactly")  
# Exactly
```

```
#Method 1 Single Statement  
while True: print(1) #infinite 1
```

```
#Method 2 Multiple Statements  
x = 0  
while x < 5: print(x); x= x + 1 # 0 1 2 3 4 5
```

Single Line Code – Continued

for loop – List of comprehension

```
#For loop in one line  
mylist = [200, 300, 400, 500]  
#Single line For loop  
result = []  
for x in mylist:  
    if x > 250:  
        result.append(x)  
print(result) # [300, 400, 500]
```

```
#One-line code way  
result = [x for x in mylist if x > 250]  
print(result) # [300, 400, 500]
```

This is list of comprehension; it creates a new list called result by iterating over each element (x) in the existing list called mylist and including only those elements that are greater than 250.

Example 13: Print the even numbers by adding 1 to the odd numbers in the list

```
odd = [1, 5, 7, 9]  
even = [i + 1 for i in odd if i % 2 == 1]  
print(even)
```

Output:
[2, 6, 8, 10]

The line of code you provided is an example of a list comprehension in Python. It creates a new list called even by iterating over each element (i) in an existing list called odd, and it includes only those elements for which the condition $i \% 2 == 1$ is true. Additionally, it increments each selected element by 1.

Single Line Code - Continued

Filtering arrays in a single line

```
mylist = [2, 3, 5, 8, 9, 12, 13, 15]
```

#Normal way

```
result = []
```

```
for x in mylist:
```

```
if x % 2 == 0:
```

```
result.append(x)
```

```
print(result) # [2, 8, 12]
```

One-line method

```
result = [x for x in mylist if x % 2 == 0]
```

```
print(result) # [2, 8, 12]
```

Single Line Code (function) - Continued

#Function in one line

#Method1

```
def fun(x): return True if x % 2 == 0 else False
```

```
print(fun(2)) # True
```

Single-line recursion

#Fibonacci Single-line recursion example

```
def Fib(x): return 1 if x in {0, 1} else Fib(x-1) + Fib(x-2)
```

```
print(Fib(5)) # 8    0, 1, 1, 2, 3, 5, 8, 13
```

```
print(Fib(15)) # 987
```

Important Questions

1. Explain the types of AI systems based on their capabilities
2. Numpy, Pandas, Matplotlib with examples and program.
3. Definitions of AI
4. Distinguish between Traditional programming and AI programming
5. When will you use the while loop? With suitable examples, explain how you use else and break statements in the while loop.
6. With suitable examples, explain how the break and continue statements are used inside a for loop.

Assignment 1

- (i) Write a function to perform sorting of given numbers and present a list of odd and even numbers separately. (15)
- (ii) Write a function to find the number of common characters in two strings. (8)
- (iii) Write a program to reverse a string. (7)

```
def sort_and_separate(numbers):  
# Sort the list in ascending order  
sorted_numbers = sorted(numbers)  
# Separate into odd and even using list comprehensions  
# Note: For decimal growth numbers, parity (odd/even) usually  
# applies to the integer part or whole integers only.  
evens = [n for n in sorted_numbers if n % 2 == 0]  
odds = [n for n in sorted_numbers if n % 2 != 0]  
return {  
    "sorted_all": sorted_numbers,  
    "even_numbers": evens,  
    "odd_numbers": odds    }
```

```
# Testing with your Population data  
population_data = [500, 700, 200, 900, 300]  
result = sort_and_separate(population_data)  
print(f"Sorted Population: {result['sorted_all']}")  
print(f"Even Populations: {result['even_numbers']}")  
print(f"Odd Populations: {result['odd_numbers']}")
```

How to Install Anaconda for Python on Your Laptop?

<https://www.youtube.com/watch?v=6zTdUoTsmzU>

(Activity / Mini Project)

Dr. K. K. THYAGHARAJAN
Professor & Dean (Research)
R.M.D. Engineering College



Python for Data Science

https://onlinecourses.nptel.ac.in/noc24_cs54/announcements?force=true

THANK YOU

THANK YOU